

Final Homework

Course of *Adaptive and Model Predictive Control*

Academic Year 2025/2026

1 Introduction and preliminary information

The present document describes the final homework for the course of *Adaptive and Model Predictive Control*. A preliminary operation that needs to be performed is to download and correctly install *acados* and its Python interface. This is necessary as the homework involves the use of this software for Nonlinear Model Predictive Control (NMPC).

The assignment is then overall composed of 4 tasks, described in Section 3. Please note that TASK 2 is composed of different points and each candidate is assigned *only* a specific one. To find out which point of TASK 2 is assigned to you, compute

$$\text{mod}(\text{YOUR STUDENT ID NUMBER}, 3) + 1$$

where $\text{mod}(a, b)$ is the *modulo operator*, returning the remainder after division of a by b .

The submission of the homework, detailed in Section 4, is due within **Tuesday 20th January 2026 at 23:59 (CET)**.

2 Setup definition

In this homework, the *inverted pendulum on a cart* system, schematically represented in Fig. 1, will be used. The variables are defined as

p	position of the cart with respect to the reference point
θ	pendulum angle with respect to the vertical direction
v	cart velocity
ω	pendulum angular velocity
F	force applied to the cart

Particularly, notice that $\theta = 0$ corresponds to the vertical *upward* position of the pendulum. The parameters of the system, together with their values are

M	mass of the cart	1 kg
m	pendulum mass	0.1 kg
ℓ	pendulum length	0.8 m
g	gravity acceleration	9.81 m/s ²

The dynamics, in the form of an explicit ODE, can be expressed as

$$\dot{p} = v, \tag{1a}$$

$$\dot{\theta} = \omega, \tag{1b}$$

$$\dot{v} = \frac{-m\ell \sin(\theta)\omega^2 + mg \cos(\theta) \sin(\theta) + F}{M + m - m \cos^2 \theta}, \tag{1c}$$

$$\dot{\omega} = \frac{-m\ell \cos(\theta) \sin(\theta)\omega^2 + F \cos(\theta) + (M + m)g \sin(\theta)}{\ell (M + m - m \cos^2(\theta))}, \tag{1d}$$

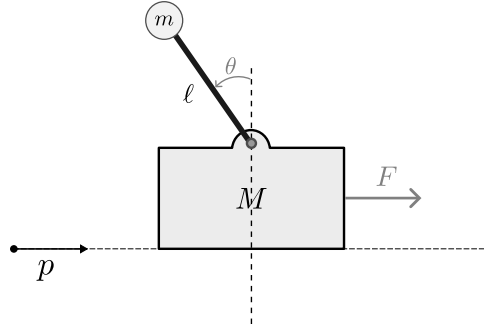


Figure 1: Schematic representation of the inverted pendulum on a cart system.

and we define as the *state* and *control input* of the system $x := [p \ \theta \ v \ \omega]^T$, $u := F$.

Furthermore, also the *linearized model* around the upward equilibrium position for the pendulum will be considered in the homework. This is given by

$$\dot{x} = \begin{bmatrix} \dot{p} \\ \dot{\theta} \\ \dot{v} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{mg}{M} & 0 & 0 \\ 0 & g \frac{M+m}{M\ell} & 0 & 0 \end{bmatrix} \begin{bmatrix} p \\ \theta \\ v \\ \omega \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \frac{1}{M} \\ \frac{1}{M\ell} \end{bmatrix} u \quad (2)$$

Concerning instead the constraints, we will consider the force that can be applied to the cart to be limited:

$$-20 \text{ N} \leq F \leq 20 \text{ N} \quad (3)$$

This system, in its original nonlinear formulation, is used in the example discussed in the lectures. This example implementation will serve as the starting point, and the overall setup described within it will also be considered. Let us briefly recall the main points:

- the system dynamics is simulated with a time step $t_{sim} = 0.001 \text{ s}$;
- the NMPC controller has sample time T_s which coincides with the amplitude of the shooting time intervals when considering a uniform time grid;
- it is possible to measure the state of the system exactly.

Remark

In the example considered in the lectures the control input is computed as

```
acados_ocp_solver.solve_for_x0(simX[i, :],
                                fail_on_nonzero_status=False,
                                print_stats_on_failure=False)
```

that is, with the flags set to False. This is because everything was previously tested, but while you are writing your own applications, be careful to check the status of the `AcadosOcpSolver` instance you are using.

If the status is 0, it indicates that the solver successfully converged to the optimal solution. If it is 2 the maximum number of iterations of the Sequential Quadratic Programming (SQP) algorithm is reached, and hence the convergence to the optimal solution is *not* achieved within the prescribed tolerance. In this situation you may still apply the computed control, but you need to be aware of this happening. Instead, other values of the status (which you can find at this [link](#)) may indicate the presence of a problem to be solved.

3 Task specifications

The tasks composing the homework are outlined in the following.

[TASK 1] Including the input rate in the formulation

Consider a *swing-up maneuver* control task, i.e. take as initial condition

$$x_0 = [0 \quad \pi \quad 0 \quad 0]^T$$

and aim at driving the state to the constant reference value

$$\bar{x} = [0 \quad 0 \quad 0 \quad 0]^T.$$

Starting from the provided example, design and test in simulation a NMPC controller to perform the *swing-up maneuver*.

You can now observe that with this design you are able to weight the *magnitude* of the control input, but not its *variation*. Therefore, what is typically done when smoother control behavior is needed is to extend the dynamics used in the controller design by introducing a new state for each control input, with its dynamics set equal to a new input computed by the controller. That is, in this case

$$\dot{p} = v, \tag{4a}$$

$$\dot{\theta} = \omega, \tag{4b}$$

$$\dot{v} = \frac{-m\ell \sin(\theta)\omega^2 + mg \cos(\theta) \sin(\theta) + F}{M + m - m \cos^2 \theta}, \tag{4c}$$

$$\dot{\omega} = \frac{-m\ell \cos(\theta) \sin(\theta)\omega^2 + F \cos(\theta) + (M + m)g \sin(\theta)}{\ell (M + m - m \cos^2(\theta))}, \tag{4d}$$

$$\dot{F} = u. \tag{4e}$$

Hence, now by weighting the input in the cost function we can penalize the variation of the force applied to the cart, while by weighting the last state we are still able to penalize its magnitude.

Implement and test this formulation in simulation. Additionally, for both this and the previous formulation, explore different weights for the cost function and compare the results obtained from one formulation with those achieved by the other.

Remark

There are a few aspects you need to pay attention to while implementing the new formulation described above:

- ▷ You need to modify only the model used inside the controller, not the simulation model.
- ▷ The control input computed by the controller will *not* be the actual input for the system, as this latter will still be the force applied to the cart.
- ▷ To understand how to compute the value of the actual input, consider that at the discrete-time instant $k = 0$ you compute u_0 starting from x_0 and F_0 , which represent the initial condition for the extended dynamics. Namely, the initial cart force F_0 needs to be defined a priori (you can set it to zero) and it will be the first control sample applied to the system. Instead, the computed input u_0 actually defines F_1 as $F_1 = F_0 + T_s u_0$, which will be applied at the next discrete-time instant and used in the initial condition for the computation of u_1 .

Therefore, in general, at the discrete-time instant k , you compute u_k starting from x_k and F_k . Then, you compute $F_{k+1} = F_k + T_s u_k$ which is used at the next iteration, and you apply F_k to the system. This means that this formulation implicitly introduces a one-step delay.

- ▷ Bounds on the magnitude of the force applied to the cart become bounds on the state.

[TASK 2] Exploring different aspects of NMPC

2.1) Numerical integration of the system dynamics

In the first part of this task, we will make use of the tools provided by *acados* only to perform numerical integration of the system dynamics (that is, *AcadosModel*, *AcadosSim* and *AcadosSimSolver* classes). Considering the original dynamics (1), use *acados* to simulate the evolution of the system for $T_f = 2.0$ s starting from the initial condition

$$x_0 = [0 \quad \pi/3 \quad 0 \quad 0]^T$$

and setting the force F always equal to zero. Use both explicit and implicit integrator types, comparing the results you obtain with both for different time steps.¹ Furthermore, consider also the average CPU time per simulation step which is employed.²

Next, modify the dynamics as follows:

$$\dot{p} = v, \tag{5a}$$

$$\dot{\theta} = \omega, \tag{5b}$$

$$\dot{v} = \frac{-m\ell \sin(\theta)\omega^2 + mg \cos(\theta) \sin(\theta) + F - kp}{M + m - m \cos^2 \theta}, \tag{5c}$$

$$\dot{\omega} = \frac{-m\ell \cos(\theta) \sin(\theta)\omega^2 + (F - kp) \cos(\theta) + (M + m)g \sin(\theta)}{\ell(M + m - m \cos^2(\theta))}, \tag{5d}$$

$$\tag{5e}$$

where $k = 10^4$ N/m. Repeat the comparison you carried out before between explicit and implicit integrators with the same sample times on this modified dynamics and $T_f = 2.0$ s, eventually commenting if there are some changes in the integrators behavior (not in that of the system, which of course will change).

Finally, let us get back to the original dynamics (1) and consider the NMPC controller for the *swing-up* maneuver with the original formulation (*not* the extended one) of TASK 1. Set $T_s = 0.1$ s, $N = 20$ (which gives a prediction horizon of $T = 2.0$ s) and test the controller using both the explicit and the implicit integrator in its design. Comment on and compare the results you obtain with respect to control performances, predicted trajectories and CPU time.

2.2) Varying Direct Multiple Shooting and SQP parameters

Consider the NMPC controller for the *swing-up* maneuver with the original formulation (*not* the extended one) of TASK 1.

Explore different shooting time interval lengths³ (which are also equal to the controller sample time in our setup) T_s and numbers of shooting intervals N , analyzing the control performances, the predicted trajectories and the total CPU time required by the controller.

Then, choose a particular combination of T_s and N which you deem most appropriate and compare the difference in the CPU time, the control performances and the predicted trajectories that you obtain by applying or not the RTI scheme.

Finally, for fixed T_s and prediction horizon length T , employ non-uniform integration grids, comparing the CPU times, the control performances and the predicted trajectories with respect to the ones obtained with the uniform integration grid.

¹As a suggestion, consider time steps in the interval $[10^{-5}, 10^{-1}]$ s.

²You can obtain this directly from the *AcadosSimSolver* object that you are using in a similar way to how it is done for the *AcadosOcpSolver* object in the provided example.

³Keep in mind that you always need $T_s = k \cdot t_{sim}$, $k \in \mathbb{N} \setminus \{0\}$.

2.3) Presence of a model mismatch

Consider the NMPC controller for the *swing-up* maneuver with the original formulation (*not* the extended one) of TASK 1.

We will now consider the case where the prediction model differs from the one used to simulate the system dynamics, not only due to a different time step for integration, but also because of an actual difference in their formulation.

To achieve this, we will insert a mismatch in the parameters of the system. For example, you can take as actual length of the pendulum $\ell_{true} = 0.9$ m, whereas the prediction model is written with $\ell = 0.8$ m.

Perform some tests with a mismatch commenting what happens in terms of control performances, particularly analyzing the predicted trajectories.

Moreover, explore different tuning of the cost function and study what happens when changing the prediction horizon length.

Remark

In inserting the mismatch, make sure to give two different values to the field `name` of the `AcadosModel` objects accounting for the simulation and the control models.

[TASK 3] Comparing NMPC with linear MPC

Even though *acados* is a software tool for Nonlinear Model Predictive Control, it can be used also for *linear* Model Predictive Control. Indeed, you can always specify in your problem formulation a *linear* dynamics instead of a nonlinear one. In such case, with the usual definition of the cost function and specifying linear constraints (like bounds), the Nonlinear Programming (NLP) problem which is obtained by Direct Multiple Shooting is actually a Quadratic Programming (QP) problem. Then, this can be solved in a single iteration of the SQP algorithm.⁴

By leveraging *acados*, design and implement a linear MPC controller using the linearized model (2), test it in simulation (using as simulation model still the nonlinear one) and then compare it against a NMPC controller which instead exploits the nonlinear model of the system. The control task to be considered is *not* the *swing-up maneuver*, but the tracking of a piecewise constant reference for the cart position, defined as

$$p_{ref}(t) = \begin{cases} 0 & 0 \text{ s} \leq t < 2.5 \text{ s} \\ 1 & 2.5 \text{ s} \leq t < 5 \text{ s} \\ -1 & 5 \text{ s} \leq t < 7.5 \text{ s} \\ 0 & 7.5 \text{ s} \leq t \leq 10 \text{ s} \end{cases} \quad (6)$$

while keeping the pendulum in the upward position. Take as initial condition

$$x_0 = [0 \ 0 \ 0 \ 0]^T.$$

Finally, discuss also whether this linear MPC controller could be used for the *swing-up* maneuver.

Notice that you have the freedom to decide what is meaningful for the comparison, and you are, of course, free to modify the design of the NMPC controller as needed.

⁴If you want, you can check this getting the number of SQP iterations by calling the `get_stats('sqp_iter')` method on the instance of `AcadosOcpSolver` class that you are using.

[TASK 4] Presenting the results

Prepare a presentation to show the obtained results, in the form of set of slides (no more than 15, with an eventual cover/title slide not included in the count). You have the flexibility to arrange the presentation as you see fit.

4 Submission details

In the dedicated repository in the *Moodle* page of the course, you must upload a single .zip file, named `yourlastname_yourstudentID.zip`, containing

- a PDF version of the presentation referenced in TASK 4;
- all files that you used, so that it is possible to replicate your experiments in case this might be needed.

Please note that the deadline for uploading the homework is an *hard deadline*, since the system will not accept any late submission.